CellAsDistribution: Decision Language Models Decode Cells as Gene-Level Distributions

Anonymous submission

Abstract

Language-model approaches to single-cell data write a cell as a list of gene names ordered by expression and generate it token by token; this offers no likelihood on counts and, as we show, discards most of what such a likelihood measures. We introduce CellAsDistribution, which treats a cell on a fixed gene panel as what its counts define, a probability distribution over genes, and fine-tunes a language model with a softmax decision head that returns this distribution from a text state in one forward pass. Cells are sampled from the decoded distribution, and every method is scored by the same exact multinomial likelihood of held-out counts. Rank sentences reconstruct log expression from rank with R^2 of 0.68 to 0.74, yet rewriting real cells as rank sentences keeps only 38% of the achievable likelihood gain on held-out CRISPRa perturbations in K562, against 64 to 73% for mass-preserving formats; an autoregressive Cell2Sentence model fine-tuned on the same backbone and budget adds no likelihood over the perturbed mean, whereas CellAsDistribution is 0.10 nats per UMI better and decodes 90 to 540 times faster. On held-out perturbations CellAsDistribution ties the strongest simple baseline in likelihood and on the top differentially expressed genes, and generates cells closer to real ones than that baseline; its per-cell gain comes from using the control cell as a residual prior on the output rather than as text in the prompt. On unseen PBMC donors it comes within 0.02 nats per UMI of the cell-type mean, the optimum for a categorical state, and on unseen cell types it generates from type names alone, beating ontology-based baselines and matching a name-matching baseline in likelihood while recovering the requested type far more often. All comparisons use leakage-controlled splits and matched control cells, new configurations were promoted by rules fixed before the test runs, and we report ties and losses with paired intervals.

1 Introduction

Predicting how a cell will behave under a new condition, such as a different cell type or a genetic perturbation, is the central task of the "virtual cell" program [1]. Experiments cover only a tiny part of the space of conditions, so a model that can generate realistic cells for unmeasured conditions would make in silico screening and hypothesis testing possible. Such a model must do more than predict an average profile: single cells from the same condition differ widely, and a faithful simulator has to reproduce this variation, which is why a generator should be judged by the likelihood it assigns to held-out cells.

Foundation models trained on millions of cells learn representations from expression tokens [2–4], but they are trained to embed cells or predict masked genes and give no likelihood for a new cell. Count models such as scVI [5] do give one, but they condition on categorical labels rather than text. Cell2Sentence (C2S) took a different route: it writes each cell as a sentence of gene names ordered by decreasing expression, so that a pretrained language model can be fine-tuned on cells and queried in natural language [6, 7]. Its case for this representation is that expression can be recovered from rank by a log-linear fit with a high R^2.

We show that this criterion is misleading for generation. A rank sentence assigns the same text to every cell with the same ordering, whatever its counts, and a language model trained on it assigns a probability to tokens rather than to a cell. On our held-out cells the C2S rank-to-expression fit indeed reaches $R^2 = 0.68$ to 0.74, yet for held-out perturbed cells the reconstructions keep only 38% of the likelihood gain that the cells' own frequency vectors achieve (Section 5.1). High R^2 on log expression and a large loss of likelihood are compatible, because the rank of a gene is easy to recover while its share of the cell's counts is not.

We therefore represent a cell by what its counts define. Fix a panel of G genes: the counts divided by their sum form a probability vector on the gene simplex, and together with the library size this vector reproduces the counts exactly (Observation 1). A decoder that outputs a point on the simplex is a generative model with an exact likelihood, and a language model becomes such a decoder when its output layer is a softmax over the G genes, a *decision head* that answers a text prompt with one probability per gene in a single forward pass [8]. We call the framework CellAsDistribution (Figure 1). We start from C2S-Scale-Gemma-2-2B [9], a Gemma-2 model [10] already trained on cell sentences, fine-tuned with low-rank adapters [11]. The idea of comparing cells as distributions over genes is not new [12], and scVI also decodes gene frequencies; what is new is a text-conditioned decoder of this distribution whose likelihood is directly comparable with that of every other method.

We evaluate CellAsDistribution where simple baselines are known to be hard to beat [13, 14]: generating cells of a given type for unseen donors (human PBMC) or of an unseen type, and generating the response to held-out CRISPRa perturbations in K562, under a gene-level split whose leakage audit found and removed a leak in an earlier version. Our contributions are:

1. **A likelihood interface for language-model cell generators.** A softmax decision head over a fixed gene panel turns a fine-tuned language model into a generator with an exact, normalized count likelihood in one forward pass, scored on the same scale as scVI and simple baselines (Section 3).

2. **A measured verdict on rank sentences.** Rank sentences look reversible by R^2 but keep 38% of the likelihood gain against 64 to 73% for mass-preserving formats, and on the same backbone and budget autoregressive Cell2Sentence adds no likelihood over the perturbed mean while CellAsDistribution gains 0.10 nats per UMI at 90 to 540 times lower decoding cost (Sections 5.1 and 5.6).

3. **Where a text decoder helps, and where it does not.** CellAsDistribution ties the strongest simple baseline on held-out perturbations; its per-cell gain comes from a control-cell residual on the output, not from reading the control cell; it generates unseen cell types from their names; and it cannot exceed the type mean for a categorical state, nor transfer knowledge to unseen genes (Sections 5.2–5.5).

4. **A protocol that exposes these differences.** Results by stratum of the split, perturbation-specific and differential-expression metrics next to likelihood, matched control cells across methods, and promotion rules written down before the test runs (Section 4).

2 Background and related work

Language models and decision language models. Autoregressive language models generate text one token at a time; a *decision language model* instead answers a typed question about a state by returning a probability over a bounded

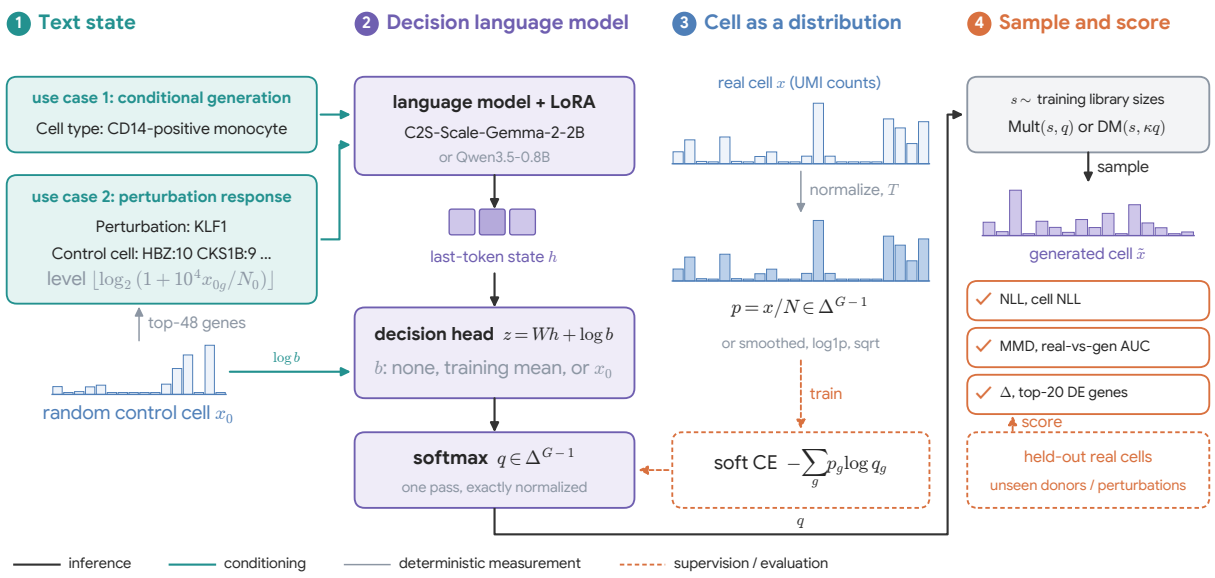


Figure 1: **A decision language model reads a text state and returns, in one forward pass, a distribution over a fixed gene panel; every method is scored with the same likelihood and sampler.** (1) The state is a prompt: a cell type (conditional generation), or a perturbation, optionally with a control cell x_0 written as its 48 most abundant panel genes with quantized log-mass levels (the tokens shown are the first ones of a real K562 control cell). (2) A LoRA-tuned language model encodes the prompt; a linear decision head on the last-token state, with an optional fixed log-prior b (none, the training mean, or the smoothed control cell itself), gives one logit per gene, and a softmax gives q on the simplex. Section 5.3 shows that the control cell is best used through $\log b$ alone. (3) A real cell is the probability vector $p = x/N$ of its counts, and training minimizes the soft cross-entropy between p and q. (4) A shared sampler draws a generated cell from q with a training library size; held-out real cells of unseen donors, perturbations or cell types score the model by likelihood, sample quality and perturbation-specific change. Bar heights are schematic.

set of options in one pass [8]. In practice this means attaching a classification-style head to the final hidden state of a pretrained model and training it on soft targets, so that the output is a calibrated distribution rather than a sampled string, which is exactly what a decoder of a gene distribution needs.

Cells as text. Cell2Sentence fine-tunes language models on rank-ordered gene sentences and has been scaled to billions of parameters and tens of millions of cells [6, 7]. Other foundation models pretrain transformers on expression tokens (scGPT, Geneformer) [2, 3] or learn zero-shot cell embeddings (UCE) [4]. BMFM-RNA reconstructs the whole expression profile from a single bottleneck token and shows that whole-cell decoding improves representations [15]. That objective is closest to ours in spirit, but it is self-supervised and its output is a continuous profile used to learn embeddings, whereas we decode a normalized simplex target from a text condition and evaluate it as a generative model.

Distributional generators of single cells. scVI models counts with a negative-binomial likelihood [5], scANVI adds cell-type labels [16], and scDiffusion samples cells with a conditional diffusion model [17]. They are the natural references for conditional generation, with likelihoods that account for overdispersion; the Gene Mover's Distance compares cells as distributions over genes [12].

Perturbation response prediction. Neural predictors such as scGen, CPA and GEARS map a perturbation to an expression change [18–20], and State predicts responses across cellular contexts [21]. A newer line models the whole perturbed population rather than a single profile: PopPert fits a low-rank Gaussian copula [22], scDFM uses conditional flow matching with an MMD objective [23], and SCALE, TempoBridge, Departures and OCOO-T learn transport or flows between control and perturbed cells [24–27]. These methods operate on continuous expression or latent vectors and are evaluated mainly with errors on mean expression, differential-expression recovery and population

distances. Independent benchmarks report that mean, additive and linear baselines are hard to beat [13, 14] and that control-referenced metrics can be inflated by systematic variation [28]. We therefore use a likelihood and the perturbed mean as references, and we compare with representative methods in Section 5.2.

3 Method

3.1 Problem setting and notation

Let G be the size of a fixed gene panel and let $x \in \mathbb{N}^G$ be the raw counts of one cell over the panel, with library size $N = \sum_g x_g$. Each cell comes with a text *state* c: a cell type for conditional generation, or a perturbation together with a control cell for perturbation response. The goal is to learn a conditional model of cells given states that can be sampled and scored. We use a panel of G=2048 genes fit on training cells only, so that no information from held-out donors or perturbations enters the representation: the genes with the largest dispersion of log-normalized expression among genes detected in more than 1% of training cells (for Norman, the training perturbation targets are always included). The selected panels contain no ribosomal-protein (RPL/RPS) or mitochondrial genes.

3.2 Why a distribution on a fixed gene panel

The panel matters for the argument that follows: once the genes are fixed, the options of the decoder are fixed too, and one can ask which representation of x keeps what a generative model needs.

Observation 1. *Let $x \in \mathbb{N}^G$ with $N = \sum_g x_g > 0$, and let $p = x/N$. (i) The map $x \mapsto (N, p)$ is injective, and a bijection onto $\{(N, p) : N \in \mathbb{N}_{>0},\ p \in \Delta^{G-1},\ Np \in \mathbb{N}^G\}$, so the frequency vector together with the library size is lossless. Moreover, if $x \sim \mathrm{Mult}(N, \pi)$, then p is the maximum-likelihood estimate of π and the multinomial log-likelihood of x under a model q is $N \sum_g p_g \log q_g +$ const, so p is all that a scoring rule needs. (ii) A rank sentence, the ordering of genes by x with a fixed tie-breaking rule, is constant on every set of count vectors that share an ordering, so it is not injective and discards all expression mass. (iii) Binarized vectors are not injective. Log-normalized or square-root vectors, once renormalized onto the simplex, define a distribution p' that differs from p unless all nonzero counts are equal, so a model trained to match p' is not fit to the multinomial likelihood of x.*

Observation 1 (proof in Appendix I) states the choice made in this paper: on a fixed panel, the frequency vector is the representation under which a count-valued cell is both fully described and directly scorable. The rank sentence of C2S is the representation that loses the most, because it keeps only the order of the genes and cannot be scored on counts without an additional reconstruction map. Log-normalized targets are used in practice to stabilize training, but after renormalization they are a different distribution from p, so training on them no longer maximizes the likelihood of the counts; Section 3.3 and Appendix E measure this trade-off. Section 5 measures the practical cost of the rank representation by rewriting real held-out cells as rank sentences, reconstructing them, and asking how much of the achievable likelihood gain survives. The same section compares two input formats for the control cell on the same backbone.

3.3 Transformations of the target

Only the training target changes with the transformation; evaluation always scores raw counts. We consider four transformations T that all land on the simplex Δ^{G-1}: the *frequency* $x/\sum x$; the *smoothed* frequency $(x+0.1)/(\sum x + 0.1G)$; *log1p*, $\log(1+x)/\sum \log(1+x)$; and *square root*, $\sqrt{x}/\sum \sqrt{x}$. Frequency is the default because it is the maximum-likelihood point for the multinomial. Log1p and square root flatten the distribution, which moves mass towards rarely expressed genes, and the smoothed target adds a uniform floor. In our experiments (Appendix E) the flattened targets cost 0.25 to 0.29 nats per UMI on Norman and worsen every sample-quality metric there, while on PBMC they trade 0.12 to 0.14 nats for a lower MMD; frequency is therefore the default throughout. An scVI-normalized target, which would reuse a latent-space decoder, was not run.

3.4 Decision language model decoder

A text state c is read by C2S-Scale-Gemma-2-2B with rank-16 LoRA adapters on all linear layers [11]. The hidden state of the last token feeds a linear *decision head* with one logit per panel gene, and a softmax gives $q_\theta(\cdot \mid c) \in \Delta^{G-1}$,

exactly normalized. Training minimizes the soft cross-entropy $-\sum_g p_g \log q_{\theta,g}$ between the transformed distribution $p = T(x)$ of a real cell and the decoded q_θ. For conditional generation, the state is the cell type. For perturbation response, the state is the perturbation name together with a *control cell* serialized as its 48 most abundant genes with a quantized log-mass level, a text that keeps mass information instead of only rank; each training perturbed cell is paired with a random control cell as in the C2S tutorial. The head can carry a fixed log-prior added to the logits: none, the training mean, or the smoothed distribution of the control cell itself, which makes the model a residual on the control cell. Because the head is initialized at zero, the model starts as exactly this prior and learns only a correction. The per-cell model we recommend, called *control residual* below, uses the control cell only through this prior and not in the prompt (Section 5.3); its language-model output is then the same for every control cell of a perturbation, and all per-cell variation enters through $\log b$. Figure 1 shows the pipeline.

Remark (what a categorical state can learn). When the state is a cell type c, the training objective averaged over the training cells $\mathcal{C}_c$ of that type is $-\sum_g \bar{p}_{c,g} \log q_g$ with $\bar{p}_c = |\mathcal{C}_c|^{-1} \sum_{i \in \mathcal{C}_c} p_i$, so by Gibbs' inequality its unique minimizer on the simplex is the equal-weight mean $\bar{p}_c$. The pooled evaluation of Section 3.5 weights cells by library size and is minimized by the UMI-weighted mean $\sum_i x_i / \sum_i N_i$. A conditional decoder can therefore at best reproduce a type mean, and its two weightings differ whenever library size covaries with composition; a loss weighted by N_i did not help on validation (-0.0013 nats per UMI, CI -0.0047 to $+0.0023$; Appendix G).

3.5 Generation and likelihood

A cell is generated by drawing a library size s from training cells and counts from $\mathrm{Mult}(s, q)$. As a second shared sampler, a Dirichlet-multinomial with concentration κ fit on training cells by the method of moments represents overdispersion, and all methods use the same sampler so that differences reflect the decoded distribution rather than the sampling step. For a perturbation, every method receives the same $M = 50$ held-out control cells, drawn with a seed derived from the perturbation's name. Held-out cells are scored on their raw counts. For the held-out cells $x_1, \ldots, x_n$ of a unit (library sizes N_i) and a model distribution q, we report the cross-entropy per UMI, $\ell(q) = -\sum_i \sum_g x_{ig} \log q_g / \sum_i N_i$. It equals the multinomial negative log-likelihood (NLL) per UMI up to the model-independent term $\sum_i [\log N_i! - \sum_g \log x_{ig}!] / \sum_i N_i$, so differences between models are exact differences in multinomial log-likelihood; we call it NLL for brevity. A model that conditions on a control cell gives one distribution q_m for each of M control cells. The pooled NLL then uses $\bar{q} = \frac{1}{M} \sum_m q_m$, and the per-cell NLL is the mixture $-\sum_i \log \left(\frac{1}{M} \sum_m \exp \sum_g x_{ig} \log q_{mg} \right) / \sum_i N_i$, up to the same constant. For each cell this lies between the score of its best-matching control and that score plus $\log M$, that is, at most $\log M / N_i$ nats per UMI above it (about 0.02 for $M{=}50$ at the median of about 216 panel UMIs per K562 cell); a model without a control cell has one q, and its two scores coincide, except for the low-rank head of Section 5.5, whose q varies from cell to cell. Because the log score is proper (Appendix I, P2), differences in NLL equal differences in KL divergence to the true expected distribution. The multinomial likelihood ignores gene-specific overdispersion, so we also report a negative-binomial log-likelihood with gene-wise dispersions fit on training cells, using the mean $s\,q$ (Appendix C); it leaves the ordering of the methods unchanged.

4 Tasks, data and protocol

Conditional generation. We use human PBMC from Hao et al. [29] (CELLxGENE Census, 2025-11-08; after removing cells labeled *unknown*, 161,159 cells of 8 donors and 29 cell types). Donors P1–P5 are used for training, P6 for validation and P7–P8 for testing, so no cell of a held-out donor is in training. The gene panel is fit on training donors only, and the units for statistics are cell types with at least 30 held-out cells.

Perturbation response. We use the CRISPRa screen of Norman et al. in K562 [30, 31] (111,445 cells; 105 single and 131 paired perturbations; 11,855 controls). In the *gene-level split*, held-out single genes occur in no training perturbation. An earlier split leaked (7/10 validation and 14/21 test single genes occurred inside training pairs); an audit found the leak and all earlier results were discarded. The 76 test perturbations comprise three strata: 21 unseen single genes, 29 pairs of one unseen and one seen gene, and 26 held-out pairs of two genes that were seen as single perturbations (validation: 10, 25 and 13); pairs of two unseen genes are excluded by construction. Results are reported overall and per stratum. We report a population-level analysis, where cells are pooled and the unit is the perturbation

(with a gene-level 5-fold over 84 single genes for power, Appendix A), and a *per-cell* analysis, where a control cell goes in and a perturbed cell comes out.

Baselines. For conditional generation we use the global mean and the cell-type mean. For perturbation response we use the control mean, the perturbed mean, a partial-additive baseline in which an unseen gene contributes nothing, ridge regression on gene embeddings, a copy of the control cell mixed with the perturbed mean (weight fit on training cells), and fine-tuned C2S following its tutorial (Section 5.1). All baselines are in frequency space and share the sampler. In addition, negative-binomial generators (scVI and scANVI, also sampled from their own negative binomial, and an scGen-style latent shift) and GEARS are compared in Sections 5.2 and 5.4, a non-language-model version of the control-cell residual isolates what the language model adds (Section 5.3), and Cell2Sentence is also fine-tuned autoregressively on the same backbone and budget as CellAsDistribution (Section 5.1); its generated sentences are mapped to frequencies by the C2S log-rank regression fit on training cells and mixed with the training perturbed mean, with the mixing weight chosen on validation from $\{0.01, 0.03, 0.1, 0.2, 0.3, 0.5, 0.7, 0.9\}$.

Metrics and statistics. We report NLL per UMI; MMD with an RBF kernel and median heuristic, and real-versus-generated AUC from a logistic classifier with 5-fold cross-validation, both in a 50-dimensional PCA space fit on training cells; the Pearson correlation of gene means (r_μ); and cell-type recovery by a classifier trained on real cells. Likelihood weights genes by their mass, and a few genes carry much of it (TMSB4X alone accounts for about 16% of panel UMIs in K562 control cells), so for perturbations we also measure the perturbation-specific change on pseudobulk profiles $\log(1 + 10^4\bar{q})$: r_Δ, the Pearson correlation between predicted and real change relative to the training perturbed centroid over all panel genes, which removes the effect shared by all perturbations [28]; and r_{DE}, the same correlation relative to the held-out control mean on the unit's top-20 differentially expressed genes (largest $|t|$ of a Welch test between its real cells and held-out control cells). The unit of analysis is the cell type or perturbation, and intervals are paired bootstraps over units. Validation and folds guide all decisions, and the test split is evaluated once per configuration and seed; for the configurations beyond the main ones, the promotion rules were written down before they were run.

5 Results

5.1 A rank sentence looks reversible but discards most of the likelihood

Objective. Cell2Sentence motivates rank sentences by showing that expression can be reconstructed from rank with a high R^2 [6]. We ask whether the same holds for the quantity a generative model is judged by, the likelihood of held-out counts.

Setup. For each held-out unit we rewrote the *real* test cells in each representation, reconstructed an expression profile from it (for rank sentences, the log-rank regression of C2S), and scored the pooled counts by multinomial NLL. The reference is the perturbed mean (Norman) or the cell-type mean (PBMC); the frequency vector of the same cells defines the achievable gain. To compare like with like, every representation, including the frequency vector, is estimated from the same subsample of cells with equal weight per cell and the same mixing weight fit on training cells; we also report the ratio against the pooled frequency oracle.

Result. By the C2S criterion (computed after the test runs) the rank sentence looks faithful: on our test cells the log-rank regression explains $R^2 = 0.68$ of log-normalized expression on Norman and 0.74 on PBMC (Pearson 0.82 and 0.87). Yet on Norman it keeps only 38% of the achievable likelihood gain (36% against the pooled oracle), whereas value bins keep 73% (69%) and our mass levels 64% (61%; Figure 2b), and validation gives the same ordering. On PBMC the representations do not separate (57–58% of the matched gain each), because differences between cell types are carried mostly by which genes are expressed rather than by how much. A high R^2 on log expression and a large loss of likelihood are therefore compatible: the rank of a gene is reconstructed well, but its share of the cell's counts is not.

The same holds for a trained model on the same backbone. To separate the representation from the backbone, we fine-tuned C2S-Scale-Gemma-2-2B autoregressively on rank sentences with the same LoRA rank, cells, epochs and batch size as CellAsDistribution, conditioned on the same control cells, and decoded its generated sentences with the C2S regression (mixing weight with the training mean chosen on validation; Section 4). The generated sentences are well formed (115 genes on average, 99.8% valid gene names), but they add no likelihood beyond the perturbed mean (-0.001 nats per UMI, CI -0.006 to $+0.005$), and the selected mixing weight is the largest on the grid (0.9). CellAsDistribution with the same backbone and budget is 0.103 nats per UMI better (CI 0.071 to 0.139) and recovers

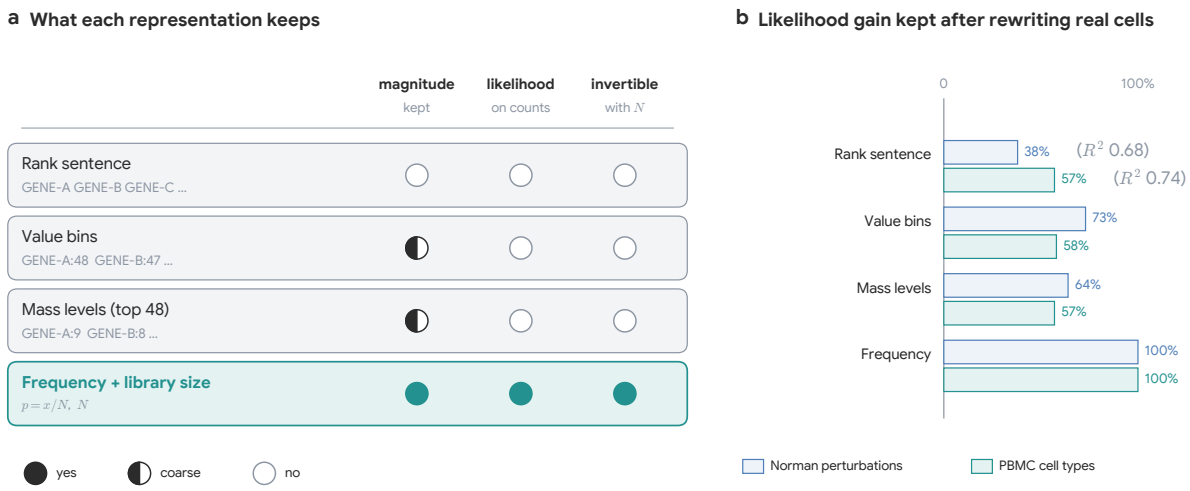


Figure 2: **A rank sentence reconstructs expression rank well but keeps little of the likelihood.** (a) What each representation keeps (Observation 1). (b) Real held-out test cells rewritten in each representation and reconstructed: share of the likelihood gain of the cells' own frequency vectors over the reference (perturbed mean for Norman, cell-type mean for PBMC), all estimated from the same cells in the same way. In grey, the criterion used to motivate rank sentences [6], the R^2 of log expression on log rank for the same reconstruction. The shares are ceilings from decoding real cells, not model results.

far more of the perturbation-specific change ($r_\Delta = 0.50$ against 0.11). The setting of the C2S perturbation tutorial (C2S-Scale-Pythia-1B, full fine-tune, 500 steps of 8 pairs) is worse still, 0.224 nats per UMI above the perturbed mean on validation (CI $+0.199$ to $+0.252$). In the reverse direction, the format of the *input* matters little once the output is a distribution: giving the control cell as a 200-gene rank sentence instead of mass levels changes the test NLL by -0.007 (CI -0.012 to $+0.000$; earlier runs, with control cells not matched across methods) at 3.7 to 4.3 times the training time.

5.2 Perturbation response: parity in likelihood, and where it comes from

Objective. Simple baselines are hard to beat on held-out perturbations [13, 14]. We ask whether CellAsDistribution matches the strongest of them in likelihood, in sample quality and in the perturbation-specific change, and in which regime of the split.

Population level (Table 1). Without a control cell, CellAsDistribution is 0.104 nats per UMI better than the perturbed mean (CI 0.071 to 0.139), ties the additive baseline (-0.009, CI -0.037 to $+0.010$) and has a better pooled NLL than the scGen-style latent shift (-0.016, CI -0.028 to -0.004). Its generated cells are closer to real ones than those of the additive baseline (MMD -0.013, CI -0.023 to -0.005; AUC -0.043, CI -0.057 to -0.029) but not than those of scGen-style, which shifts every control cell individually. GEARS, trained with its default settings on the same split, is 0.466 nats above the perturbed mean; its output is a regression of log-normalized expression that we map back to frequencies, and we did not tune it, so this row measures that conversion as much as the model.

By stratum (Table 2). The likelihood gain is concentrated where training contains the genes: on held-out pairs CellAsDistribution is 0.108 (one unseen gene) and 0.178 (both genes seen) nats better than the perturbed mean, whereas on unseen single genes it ties it (-0.006, CI -0.025 to $+0.018$). Likelihood is dominated by high-mass genes, and the perturbation-specific metrics separate the methods differently: on pairs, CellAsDistribution's predicted change correlates slightly less with the real one than the additive baseline's over all panel genes (r_Δ lower by about 0.05), while on each perturbation's top-20 DE genes it ties additive (r_{DE} 0.84 and 0.90) and exceeds scGen-style. The decoder thus reproduces the dominant transcriptional changes of known genes as well as additive composition does, but it does not yet capture the diffuse genome-wide part of the effect as well.

Gene names, not gene knowledge. Replacing gene symbols by opaque identifiers worsens the overall NLL ($+0.014$, CI $+0.002$ to $+0.027$) and lowers r_Δ in every stratum, as much on pairs of seen genes (-0.043, CI -0.067 to -0.020) as on unseen single genes (-0.067, CI -0.126 to -0.007), so names help the model organize what it

Model	NLL ↓ (Δ to additive)	cell NLL ↓	MMD ↓	AUC ↓
Perturbed mean	5.289 (+0.094 [+0.058, +0.136])	5.289	0.0726	0.903
Additive (reference)	5.195	5.195	0.0519	0.855
scGen-style	5.201 (+0.006 [-0.014, +0.022])	5.105	**0.0286**	**0.769**
GEARS	5.755 (+0.561 [+0.518, +0.603])	5.755	0.1982	0.980
C2S, autoregressive (Gemma)	5.288 (+0.094 [+0.059, +0.133])	5.279	0.0752	0.906
Additive, control residual (no LM)	5.199 (+0.005 [-0.001, +0.011])	5.134	0.0381	0.815
CellAsDistribution, no control cell	**5.186 (-0.009 [-0.037, +0.010])**	5.186	0.0390	0.812
CellAsDistribution, control residual	5.197 (+0.002 [-0.025, +0.022])	5.129	0.0300	0.785
CellAsDistribution, NB head	5.201 (+0.006 [-0.021, +0.028])	5.201	0.0365	0.797
CellAsDistribution, low-rank head	5.247 (+0.053 [+0.024, +0.084])	**5.096**	0.0401	0.791

Table 1: Perturbation response, Norman K562, test split (76 held-out perturbations; matched control cells; seed 0; C2S-Scale-Gemma-2-2B backbone for every language model). NLL: cross-entropy per UMI of the pooled real cells (paired difference to the additive baseline with 95% bootstrap interval over perturbations). cell NLL: mixture over the $M = 50$ control cells (or the low-rank head's draws). MMD and real-versus-generated AUC on cells from the shared Dirichlet-multinomial sampler. Best value in bold; CellAsDistribution rows shaded. Per-stratum results in Table 2.

	ΔNLL to additive ↓			r_Δ ↑		r_{DE} ↑	
Model	single ($n = 21$)	1 unseen ($n = 29$)	both seen ($n = 26$)	1 unseen	both	1 unseen	both
Additive	0	0	0	0.550	0.643	0.843	0.910
scGen-style	-0.002 [-.015,+.011]	+0.037 [+.023,+.053]	-0.021 [-.072,+.013]	0.497	0.627	0.804	0.875
C2S, autoregressive	+0.005 [+.003,+.008]	+0.117 [+.059,+.184]	+0.139 [+.062,+.220]	0.102	0.093	0.541	0.657
Additive, control residual	-0.004 [-.010,+.003]	+0.017 [+.007,+.029]	-0.002 [-.013,+.008]	0.536	0.635	0.829	0.909
CellAsDistribution, no control cell	-0.006 [-.025,+.018]	+0.010 [-.007,+.026]	-0.033 [-.103,+.010]	0.498	0.596	0.841	0.901
CellAsDistribution, control residual	+0.005 [-.019,+.036]	+0.027 [+.011,+.047]	-0.028 [-.098,+.015]	0.464	0.571	0.805	0.895
CellAsDistribution, NB head	-0.016 [-.029,-.002]	+0.029 [+.009,+.051]	-0.002 [-.076,+.060]	0.507	0.605	0.840	0.893
CellAsDistribution, opaque gene ids	+0.001 [-.013,+.015]	+0.029 [+.011,+.049]	-0.019 [-.096,+.031]	0.402	0.553	0.754	0.863

Table 2: Norman test split by stratum of the gene-level split. ΔNLL: paired difference to the additive baseline [95% bootstrap over perturbations]. r_Δ: Pearson correlation of predicted and real change relative to the training perturbed centroid over all panel genes; r_{DE}: the same relative to the held-out control mean on each perturbation's top-20 DE genes (pseudobulk). On unseen single genes the additive baseline equals the perturbed mean, so r_Δ is not reported there.

learns from training perturbations; on unseen single genes the likelihood does not change ($+0.007$, CI -0.022 to $+0.030$). Adding NCBI gene summaries to the prompt did not help on validation. As in the population-level analysis (Appendix A), we find no evidence that the language model transfers knowledge to genes it has not seen perturbed. One visible reason is biological: CRISPRa raises the expression of its own target, which for the median unseen single gene, MAML2, is the most strongly increased of its top-20 DE genes (Figure 4b), and a decoder that has never seen the gene perturbed has no way to infer this from the name alone.

5.3 The per-cell gain comes from the control-cell residual

Objective. With the log-prior $b = \log \mathrm{smooth}(x_0)$, the zero-initialized head outputs exactly the control cell at the start of training and learns only a correction (Section 3.4). We ask whether the per-cell gain comes from this residual, from the language model reading the control cell as text, or from both.

Result. Removing the control-cell text while keeping the residual improves the per-cell NLL by 0.042 (CI 0.035 to 0.050) and lowers MMD and AUC (Table 1); keeping the text but removing the residual loses the per-cell gain ($+0.019$, CI $+0.012$ to $+0.027$). The residual model is 0.066 nats per cell better than the additive baseline (CI 0.045 to 0.095) and matches scGen-style on MMD ($+0.001$, CI -0.003 to $+0.005$), although scGen-style keeps a lower per-cell NLL (0.024, CI 0.016 to 0.032). A version of the same residual without a language model (added after the test runs), in which the additive shift replaces the learned correction, reaches the same likelihood (pooled -0.002, CI -0.028 to $+0.016$; per cell -0.005, CI -0.029 to $+0.010$), but its cells are less realistic (MMD $+0.008$, CI $+0.001$ to $+0.018$; AUC $+0.030$, CI $+0.013$ to $+0.047$, both relative to CellAsDistribution). The residual construction thus accounts for

Model	NLL ↓ (Δ to type mean)	cell NLL ↓	MMD ↓	AUC ↓	type acc. ↑
Global mean	6.732 (+1.125 [+0.939, +1.326])	6.732	0.8329	1.000	0.038
Cell-type mean (reference)	5.606	5.606	0.1816	0.985	0.879
scVI	5.611 (+0.005 [-0.007, +0.016])	5.611	0.1659	0.975	0.824
scVI, own NB sampling	5.605 (-0.001 [-0.011, +0.010])	5.489	0.1284	0.924	0.695
scANVI, own NB sampling	5.610 (+0.003 [-0.009, +0.017])	5.486	0.1303	0.930	0.666
CellAsDistribution (Gemma)	5.623 (+0.017 [+0.001, +0.036])	5.623	0.1630	0.970	0.815
CellAsDistribution (Qwen)	5.636 (+0.030 [+0.001, +0.073])	5.636	0.1717	0.972	0.815
CellAsDistribution, low-rank head	5.647 (+0.041 [+0.018, +0.067])	5.552	0.1654	0.970	0.769

Table 3: Conditional generation, human PBMC, test split (unseen donors P7 and P8; 24 cell types with at least 30 cells; seed 0). NLL: paired difference to the cell-type mean [95% bootstrap over cell types]. *Own NB sampling*: cells drawn from scVI/scANVI's own negative binomial from latent states of training cells of the type, and cell NLL as the mixture over these draws; all other rows use the shared samplers. Generation metrics on Dirichlet-multinomial samples except for the own-sampling rows.

Model (unseen cell type)	ΔNLL to global mean ↓	ΔNLL to name-overlap ↓	r_μ ↑	type acc. ↑
Nearest Cell Ontology type	-0.209 [-0.612, +0.328]	+0.440 [+0.133, +0.860]	0.682	0.018
Cell Ontology parent	-0.515 [-0.737, -0.285]	+0.134 [+0.001, +0.299]	0.745	0.050
Name-overlap type (string match)	-0.649 [-0.877, -0.388]	0	0.793	0.022
CellAsDistribution, opaque type id	+0.106 [-0.154, +0.377]	+0.754 [+0.470, +1.041]	0.516	0.009
CellAsDistribution, type name	-0.707 [-0.875, -0.542]	-0.059 [-0.216, +0.097]	0.787	0.146
CellAsDistribution, name + CL definition	-0.707 [-0.862, -0.545]	-0.058 [-0.219, +0.101]	0.789	0.138

Table 4: Zero-shot cell types, PBMC test split (unseen donors): 19 cell types, each held out of training in one of five leave-types-out folds (Qwen3.5-0.8B backbone, seed 0). Paired differences [95% bootstrap over cell types]. Baselines use only training types: the type closest in the Cell Ontology, the pooled training types under the held-out type's Cell Ontology parents, and the training type whose name shares the most words.

the per-cell likelihood gain, and what the language model adds on top is realism of the generated cells. The control cell is best used as a prior on the output rather than as text in the prompt: neither text form improves on the residual alone (per-cell NLL 5.171 with mass-level text, 5.147 with genes over-represented relative to the average control, 5.129 without text).

5.4 Conditional generation: a type mean for known types, composition of names for unseen ones

Objective. When the state is only a cell type, the Remark in Section 3.4 says that the best a decoder can learn is the type mean. We test this on unseen donors, and then ask what the text interface buys when the type itself is unseen.

Known types (Table 3). As predicted, no model improves on the cell-type mean in likelihood: CellAsDistribution is slightly worse ($+0.017$ nats per UMI, CI $+0.001$ to $+0.036$), scVI and scANVI tie it, and on validation CellAsDistribution gives the same NLL to four decimals for every LoRA rank and, at the same seed, for both backbones. On sample quality the decisive factor is the source of cell-to-cell variation, not the decoder: scVI and scANVI sampling from their own negative binomial from latent states of training cells reach the lowest MMD (0.128) and AUC (0.924) and a lower mixture NLL (5.489 against 5.606), whereas every frequency decoder, ours included, relies on the shared sampler. A low-rank head that lets CellAsDistribution sample its own variation lowers its mixture NLL to 5.552 but does not reach scVI. No generator is close to indistinguishable from real cells of unseen donors (AUC ≥ 0.92; UMAP views in Appendix F).

Unseen types (Table 4). A model indexed by cell type cannot generate a type it has never seen; a decoder that reads the type's name can. In five leave-types-out folds over the 19 PBMC types with enough cells, CellAsDistribution trained on the remaining types generates each held-out type from its name and formal Cell Ontology definition (the pre-registered configuration). It is 0.707 nats per UMI better than the global mean (CI 0.545 to 0.862), 0.498 better than the nearest training type in the Cell Ontology (CI 0.146 to 0.984), and 0.192 better than the pooled training types

under the held-out type's ontology parents (CI 0.045 to 0.352; this and the next baseline were added after the test runs). It ties a string-matching baseline that copies the training type whose name shares the most words (-0.058, CI -0.219 to $+0.101$), but it recovers the requested type far more often (0.138 against 0.022). The name alone gives the same likelihood as name plus definition (difference below 0.001 nats), and replacing names by opaque identifiers removes the gain entirely. The decoder therefore generalizes by composing the words of type names; we find no evidence that it uses knowledge beyond the names.

5.5 Distributional heads: each buys one property at the cost of another

Objective. A flat softmax gives one distribution per state, so cell-to-cell variation and gene-specific dispersion come only from the shared sampler (Section 3.5). We tested two heads that keep the exact normalization of q and the single language-model pass, each promoted from validation by the pre-written rules.

Negative-binomial head. The head adds a per-gene inverse dispersion $\theta_g(c)$ and is trained on the negative-binomial likelihood of the counts with mean $N_i q_g$, so genes with high mass no longer dominate the gradient. On the Norman test split (no control cell, against the softmax head) it raises the perturbation-specific correlation r_Δ by 0.020 (CI 0.012 to 0.029), and by 0.049 on unseen single genes (CI 0.034 to 0.067), with a slightly lower MMD and AUC, at a small cost in pooled multinomial NLL ($+0.015$, CI $+0.006$ to $+0.028$). On PBMC, where the state is categorical, it did not help on validation and was not promoted.

Low-rank logistic-normal head. The logits become $z = Wh + \log b + U\,\mathrm{diag}(a(h))\,\epsilon$ with $\epsilon \sim \mathcal{N}(0, I_{16})$, so q itself varies from cell to cell along 16 learned gene programs; training uses a variational bound with an encoder of the target cell, and generation draws ϵ from the prior. Without seeing any control cell, it lowers the per-cell NLL by 0.089 (CI 0.080 to 0.098) on Norman, and on PBMC from 5.623 to 5.552 (Table 3), reaching the per-cell NLL of the scGen-style model, which encodes every control cell (-0.009, CI -0.019 to $+0.002$). The price is a worse pooled NLL of the mean over draws ($+0.062$, CI $+0.040$ to $+0.087$) and a lower r_Δ (-0.067, CI -0.092 to -0.042). The per-cell NLL is estimated from 50 (Norman) or 300 (PBMC) prior draws per state; this log-mean-exp estimator is biased upwards, so the gain is if anything understated.

5.6 Cost and robustness

One pass instead of hundreds of tokens. Because the head returns the whole distribution at once, decoding costs one forward pass of the prompt. Measured after the test runs, alone on one A100 with the same backbone and batch size (25), C2S-Gemma needs 1.99 seconds to generate one cell sentence (530 new tokens on average), whereas CellAsDistribution needs 0.004 seconds per cell with the control cell as a residual and 0.022 seconds when the control cell is also written as 342 tokens of text, that is, 90 to 540 times less; without a control cell, one pass of 0.004 seconds gives the distribution for every cell of that state. Training on the same cells is also cheaper: 13 minutes for CellAsDistribution against 128 minutes for C2S-Gemma, although these wall-clock times were measured with other jobs on the GPU.

Backbone, seeds and adapter rank. We compare the general-purpose Qwen3.5-0.8B with the Cell2Sentence-initialized C2S-Scale-Gemma-2-2B [9, 10] under the same head, LoRA rank and budget, with three seeds each on test. Gemma is consistently but modestly better: the pooled NLL is 5.6272 against 5.6394 for conditional generation and 5.1848 against 5.1958 for perturbation response without a control cell, a gap of 0.011 to 0.012 nats that is present for every seed and exceeds the seed standard deviation (0.001 to 0.003), yet does not change any ranking against the baselines. On validation the backbone made no measurable difference, and LoRA ranks 8, 16 and 64 give the same NLL to within 0.002 nats. Across seeds the control-residual model varies by at most 0.004 in per-cell NLL (5.127 to 5.130).

6 Conclusion and discussion

Writing a cell as what its counts define, a distribution over a fixed panel, turns a language model into a cell generator with an exact likelihood: a softmax decision head returns the whole distribution in one pass, and the same multinomial score applies to every method. This interface makes the representation a measurable choice. Rank sentences look reversible by the criterion used to motivate them, yet they lose most of the likelihood that a distribution keeps, and on the same backbone and budget an autoregressive rank-sentence model adds no likelihood over the perturbed mean while costing two orders of magnitude more to decode. As a model, CellAsDistribution is as good as the strongest simple baseline on held-out perturbations, generates more realistic cells than that baseline when the control cell enters as a

residual prior, and generates unseen cell types from their names. Its limits are equally clear and, we argue, informative: a categorical state can only reach its training mean, the language model's gene knowledge does not transfer to genes it has not seen perturbed, and the diffuse genome-wide part of a perturbation is still captured better by additive composition. These limits point to what the next text-conditioned cell model needs: richer states than a label (donor, cell-level and pathway context), output heads that model cell-to-cell covariance and dispersion together, as our negative-binomial and low-rank heads begin to do separately, and evaluation by stratum and by perturbation-specific change rather than by a single pooled number.

Limitations. There is one dataset per task, a 2048-gene panel, and CRISPRa genetic perturbations only; drug and dose responses were not studied. Held-out control cells are not paired with the perturbed cells, so per-cell scores reward covering the distribution of cell states rather than predicting a counterfactual. The multinomial likelihood ignores gene-specific overdispersion (a negative-binomial evaluation leaves the rankings unchanged, Appendix C), and the flat head models neither dispersion nor gene–gene covariance; the two new heads each address one of these at a cost. GEARS was run with default settings and its regression output converted to frequencies, and CPA and the transport and flow models of Section 2 were not compared, because they do not output a distribution over a fixed gene list. Zero-shot cell types were evaluated with the Qwen backbone and five folds over 19 types. Each test configuration was evaluated once per seed: Qwen seed 0 was first evaluated before the Gemma runs, Gemma was chosen as the main backbone before its test results were known, the configurations beyond the main ones were promoted by rules written down before their test runs (Appendix G), and three zero-shot validation runs that failed for lack of memory were re-run after the zero-shot gate had been applied to the remaining two folds; on all five folds the gate gives the same decision.

Code availability. Code, held-out unit lists, run scripts and the table-generation scripts are available, with a README, at https://anonymous.4open.science/r/CellAsDistribution-R27.

AI use statement. Code, experiments, tables, figures and a first draft of this text were produced with assistance from Claude (Anthropic); the authors reviewed all content and are responsible for it.

References

[1] Charlotte Bunne, Yusuf Roohani, Yanay Rosen, et al. How to build the virtual cell with artificial intelligence: Priorities and opportunities. *Cell*, 187:7045–7063, 2024. doi: 10.1016/j.cell.2024.11.015.

[2] Haotian Cui et al. scGPT: toward building a foundation model for single-cell multi-omics using generative AI. *Nature Methods*, 21:1470–1480, 2024. doi: 10.1038/s41592-024-02201-0.

[3] Christina V. Theodoris et al. Transfer learning enables predictions in network biology. *Nature*, 618:616–624, 2023. doi: 10.1038/s41586-023-06139-9.

[4] Yanay Rosen, Yusuf Roohani, Ayush Agrawal, et al. Universal cell embedding provides a foundation model for cell biology. *Nature*, 656(8126):183–191, 2026. doi: 10.1038/s41586-026-10689-z.

[5] Romain Lopez, Jeffrey Regier, Michael B. Cole, Michael I. Jordan, and Nir Yosef. Deep generative modeling for single-cell transcriptomics. *Nature Methods*, 15:1053–1058, 2018. doi: 10.1038/s41592-018-0229-2.

[6] Daniel Levine, Syed A. Rizvi, Sacha Lévy, et al. Cell2Sentence: Teaching large language models the language of biology. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 27299–27325, 2024. URL https://proceedings.mlr.press/v235/levine24a.html.

[7] Syed Asad Rizvi, Daniel Levine, Aakash Patel, et al. Scaling large language models for next-generation single-cell analysis. bioRxiv, 2025.

[8] Unsloth. Train your own decision model with Unsloth. Documentation, 2026. URL https://unsloth.ai/docs/basics/train-your-own-decision-model-with-unsloth.

[9] van Dijk Lab. C2S-Scale-Gemma-2-2B (model card). Hugging Face, vandijklab/C2S-Scale-Gemma-2-2B, 2025. URL https://huggingface.co/vandijklab/C2S-Scale-Gemma-2-2B.

[10] Gemma Team. Gemma 2: Improving open language models at a practical size. arXiv:2408.00118, 2024.

[11] Edward J. Hu, Yelong Shen, Phillip Wallis, et al. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. arXiv:2106.09685.

[12] Riccardo Bellazzi, Andrea Codegoni, Stefano Gualandi, Giovanna Nicora, and Eleonora Vercesi. The Gene Mover's Distance: Single-cell similarity via optimal transport. arXiv:2102.01218, 2021.

[13] Constantin Ahlmann-Eltze, Wolfgang Huber, and Simon Anders. Deep-learning-based gene perturbation effect prediction does not yet outperform simple linear baselines. *Nature Methods*, 22:1657–1661, 2025. doi: 10.1038/s41592-025-02772-6.

[14] Zhiting Wei, Yiheng Wang, Yicheng Gao, et al. Benchmarking algorithms for generalizable single-cell perturbation response prediction. *Nature Methods*, 23:451–464, 2026. doi: 10.1038/s41592-025-02980-0.

[15] Michael M. Danziger, Bharath Dandala, Viatcheslav Gurev, et al. BMFM-RNA: whole-cell expression decoding improves transcriptomic foundation models. arXiv:2506.14861, 2025.

[16] Chenling Xu, Romain Lopez, Edouard Mehlman, et al. Probabilistic harmonization and annotation of single-cell transcriptomics data with deep generative models. *Molecular Systems Biology*, 17(1):e9620, 2021. doi: 10.15252/msb.20209620.

[17] Erpai Luo, Minsheng Hao, Lei Wei, et al. scDiffusion: conditional generation of high-quality single-cell data using diffusion model. *Bioinformatics*, 40(9):btae518, 2024. doi: 10.1093/bioinformatics/btae518.

[18] Mohammad Lotfollahi, F. Alexander Wolf, and Fabian J. Theis. scGen predicts single-cell perturbation responses. *Nature Methods*, 16:715–721, 2019. doi: 10.1038/s41592-019-0494-8.

[19] Mohammad Lotfollahi, Anna Klimovskaia Susmelj, Carlo De Donno, et al. Predicting cellular responses to complex perturbations in high-throughput screens. *Molecular Systems Biology*, 19(6):e11517, 2023. doi: 10.15252/msb.202211517.

[20] Yusuf Roohani, Kexin Huang, and Jure Leskovec. Predicting transcriptional outcomes of novel multigene perturbations with GEARS. *Nature Biotechnology*, 42:927–935, 2024. doi: 10.1038/s41587-023-01905-6.

[21] Abhinav K. Adduri, Dhruv Gautam, Beatrice Bevilacqua, et al. Predicting cellular responses to perturbation across diverse contexts with State. *Cell*, 189(19):5914–5931.e20, 2026. doi: 10.1016/j.cell.2026.07.052.

[22] Handong Wang, Jiaxin Qi, Haochen Feng, and Baisheng Lai. PopPert: Population-level joint-distribution modeling for single-cell perturbation prediction. arXiv:2609.01357, 2026.

[23] Chenglei Yu, Chuanrui Wang, Bangyan Liao, and Tailin Wu. scDFM: Distributional flow matching model for robust single-cell perturbation prediction. In *International Conference on Learning Representations*, 2026. arXiv:2602.07103.

[24] Shuizhou Chen, Lang Yu, Xueqin Lin, et al. SCALE: Scalable conditional atlas-level endpoint transport for virtual cell perturbation prediction. arXiv:2603.17380, 2026.

[25] Bowen Han, Lingbei Meng, Shihuan Luo, et al. TempoBridge: Source-conditioned flow matching with optimal transport couplings for single-cell population transitions. arXiv:2610.04945, 2026.

[26] Changxi Chi, Yufei Huang, Jun Xia, et al. Departures: Distributional transport for single-cell perturbation prediction with neural Schrödinger bridges. arXiv:2511.13124, 2025.

[27] Danning Jiang, Zhiwen Yan, Qirun Wang, et al. OCOO-T: A simple and scalable virtual cell model for transcriptional perturbation response prediction. arXiv:2606.12838, 2026.

[28] Ramon Viñas Torné, Maciej Wiatrak, Zoe Piran, et al. Systema: a framework for evaluating genetic perturbation response prediction beyond systematic variation. *Nature Biotechnology*, 44(6):1050–1059, 2026. doi: 10.1038/s41587-025-02777-8.

[29] Yuhan Hao et al. Integrated analysis of multimodal single-cell data. *Cell*, 184:3573–3587.e29, 2021. doi: 10.1016/j.cell.2021.04.048.

[30] Thomas M. Norman, Max A. Horlbeck, Joseph M. Replogle, et al. Exploring genetic interaction manifolds constructed from rich single-cell phenotypes. *Science*, 365:786–793, 2019. doi: 10.1126/science.aax4438.

[31] Stefan Peidli, Tessa D. Green, Ciyue Shen, et al. scPerturb: harmonized single-cell perturbation data. *Nature Methods*, 21:531–540, 2024. doi: 10.1038/s41592-023-02144-y.

Supplementary material

CellAsDistribution: Decision Language Models Decode Cells as Gene-Level Distributions

A K-fold analysis on Norman

This analysis was run with the Qwen3.5-0.8B backbone (CAD-P, the flat decoder of the population-level analysis) before the Gemma backbone was added. Pooled over five gene-level folds, the flat decoder ties the partial-additive baseline on pairs of seen genes (-0.017, CI -0.060 to $+0.021$, $n = 28$) and on pairs with one unseen gene ($+0.004$, CI -0.014 to $+0.022$, $n = 51$), and it beats ridge. The split-half oracle lies 0.108 nats below the perturbed mean on unseen single genes, so headroom exists; descriptions and shuffled names do not help, and shuffled names are harmful ($+0.045$). A one-parameter target-induction baseline, which raises the unseen target gene's frequency, is better than every learned decoder on the 75 genes whose target is in the panel (-0.013, CI -0.031 to -0.004). On the test split, CAD-P (Qwen backbone) is 0.097 better than the perturbed mean and ties additive (-0.003, CI -0.031 to $+0.019$).

B Training details

Table 6 lists the training settings of the decision language models, and the paragraphs below it describe the scVI-based baselines.

scVI and scANVI. Both are trained on training cells of the panel with a negative-binomial likelihood, 30 latent dimensions and 60 epochs (scANVI: 15 further epochs with cell-type labels). The frequency vector of a cell type is the mean decoder output over up to 3000 training cells of that type.

scGen-style baseline. An scVI model is trained on all training cells. The latent shift of a single gene is the mean latent state of its perturbed cells minus that of the control cells, the shift of a pair is the sum of the shifts of its known genes, and a perturbation with no known gene uses the mean shift of all training perturbations. Each held-out control cell is encoded, shifted and decoded with library size one.

C Negative-binomial evaluation

For a held-out cell i of a unit with library size s_i and model frequency q, the count of gene g is scored as $\mathrm{NB}(x_{ig} \mid \mu = s_i q_g, \phi_g)$ with variance $\mu + \phi_g \mu^2$. The dispersions ϕ_g come from a method-of-moments fit on library-normalized training counts with the within-unit variance pooled over units (median $\phi = 26.5$ for PBMC and 9.0 for Norman, clipped to $[10^{-3}, 10^2]$). The score is the negative log-likelihood per UMI, which is on a different scale from the multinomial score because it includes the zeros, so only differences are compared (Table 7). It uses the pooled frequency of a unit, so it does not reward conditioning on a control cell.

On PBMC the negative-binomial score separates the decoders slightly more than the multinomial score does: CAD is reliably, but marginally (0.004 nats), worse than the cell-type mean. On Norman the ordering of the methods is the same as under the multinomial score.

D Worked examples: prompts and decoded distributions

Figure 4 shows, for every use case, the exact prompt that the decision language model reads and the distribution it returns, next to the held-out ground truth. The examples are not hand-picked. In each case, the shown unit is the held-out cell type or perturbation whose paired NLL difference between CellAsDistribution and the case's reference is the median over the units of that case, so half of the units look better and half look worse. The cases are:

- conditional generation on unseen PBMC donors, against the cell-type mean;
- perturbation response without a control cell, one example per stratum of the gene-level split (unseen single gene, pair with one unseen gene, pair of two seen genes), against the additive baseline;

Paired NLL/UMI difference [95% CI]	unseen singles ($n = 84$)	pairs (seen genes) ($n = 28$)	pairs (one unseen gene) ($n = 51$)
to the perturbed mean			
additive (partial)	+0.000 [+0.000, +0.000]	-0.127 [-0.186, -0.077]	-0.076 [-0.111, -0.044]
ridge	**-0.002 [-0.010, +0.006]**	-0.033 [-0.053, -0.013]	-0.015 [-0.025, -0.006]
CAD-P, real gene names	+0.007 [-0.010, +0.023]	-0.144 [-0.197, -0.095]	-0.072 [-0.101, -0.046]
CAD-P, opaque ids	+0.015 [+0.002, +0.031]	-0.147 [-0.208, -0.099]	-0.066 [-0.090, -0.042]
CAD-P, shuffled names	+0.045 [+0.027, +0.063]	-0.155 [-0.208, -0.108]	-0.053 [-0.079, -0.027]
CAD-P + own description	+0.007 [-0.008, +0.022]	**-0.157 [-0.211, -0.112]**	**-0.084 [-0.112, -0.057]**
CAD-P + other gene's description	+0.018 [+0.003, +0.033]	-0.153 [-0.197, -0.114]	-0.072 [-0.102, -0.045]
split-half oracle (reference)	-0.108 [-0.147, -0.075]	-0.155 [-0.215, -0.105]	-0.151 [-0.188, -0.118]
between decoders			
real − opaque ids (primary)	-0.008 [-0.029, +0.011]	+0.003 [-0.019, +0.030]	-0.006 [-0.025, +0.013]
real − shuffled names	-0.038 [-0.061, -0.016]	+0.011 [+0.000, +0.025]	-0.019 [-0.046, +0.004]
description − other description	-0.011 [-0.022, -0.001]	-0.003 [-0.024, +0.012]	-0.012 [-0.022, -0.000]
description − real names	+0.000 [-0.010, +0.011]	-0.013 [-0.026, -0.002]	-0.012 [-0.024, -0.000]
CAD-P − additive	+0.007 [-0.010, +0.023]	-0.017 [-0.060, +0.021]	+0.004 [-0.014, +0.022]
CAD-P − ridge	+0.009 [-0.008, +0.025]	-0.111 [-0.167, -0.062]	-0.057 [-0.084, -0.033]

Table 5: Gene-level 5-fold analysis on Norman (CAD-P, Qwen3.5-0.8B backbone): pooled paired NLL differences (nats per UMI; negative favors the first term). Unit: perturbation.

Setting	Value
Optimizer, schedule	AdamW (weight decay 0), one-cycle schedule with 5% warm-up, gradient clipping at 1.0
Learning rates	2×10^{-4} (LoRA adapters), 10^{-3} (decision head, zero-initialized)
Batch size, budget	16; one epoch over 20,000 training cells drawn at random (1,250 steps)
LoRA	rank 16, $\alpha = 16$, dropout 0, all linear layers; rank 8 and 64 for the sensitivity run
Precision, attention	bf16; eager attention and gradient checkpointing for Gemma-2
Output vocabulary	the 2048 panel genes as classes; no new tokens, gene names are tokenized as plain text
Control-cell text	48 most abundant genes with 14 log-mass levels; 200 gene names for the rank variant
Seeds	test: Gemma and Qwen 0, 1, 2; validation: Gemma 3407 (default), 1, 2; Qwen 1, 2; variants of Appendix G: Qwen seed 0 (the seed changes the sampled training cells and the sampler)
Negative-binomial head	inverse dispersion $\log \theta_g(c) = \beta_g + v_g^\top h$, initialized at $\theta = 1$ and clipped to $[e^{-6}, e^{8}]$; loss: negative-binomial NLL of the raw panel counts with mean $N_i q_g$, per UMI
Low-rank head	$k = 16$ programs; amplitude $a(h) = \mathrm{softplus}(\cdot)$ initialized at 1; encoder of the target cell: $\log(1 + 10^4 x/N) \to 256$ (GELU) $\to 2k$; one reparameterized sample per cell; KL weight raised linearly from 0 to 1 over the first 30% of steps
C2S-Gemma baseline	C2S-Scale-Gemma-2-2B, LoRA $r = 16$ on all linear layers, lr 2×10^{-4}, one-cycle schedule, 20,000 cells, one epoch, batch 16 (micro-batches of 2); control and target as top-200 rank sentences; loss on the response only; sampling with top-k 50, top-p 0.95, up to 1024 new tokens
Evaluation	$M = 50$ held-out control cells per perturbation, the same for every method (earlier runs: drawn separately per method); 300 generated cells per unit
Hardware, time	one A100 40 GB; run alone, Qwen takes 6 to 8.5 min and Gemma 9.5 to 24 min per run (90 to 100 min with the 200-gene rank text; wall-clock was longer when jobs shared the GPU); Gemma peak memory 8.7 GB

Table 6: Training settings of the decision language models.

- perturbation response with a control cell, for a pair of two seen genes, once with the control cell both written as mass-level text in the prompt and used as the residual prior (the prompt shows a real serialized control cell), and once with the control cell used only as the residual prior;
- zero-shot cell types (fold 0, so the median of its 4 held-out types), against the mean of the nearest training type in the Cell Ontology.

The ground truth is the pooled frequency of all held-out real cells of the unit. For a control-cell model the plotted distribution is the mean over the $M = 50$ matched control cells, and the prompt shown is the one built from the first of them. For perturbations, the right panel compares the log fold change over the held-out control mean on the unit's top-20 differentially expressed genes (largest $|t|$ of a Welch test between its real cells and the held-out control cells, computed on real cells only). The selected units and the numbers quoted for them are:

(a) *PBMC, conditional generation*: naive thymus-derived CD4-positive, alpha-beta T cell, the median of 24 units (NLL difference to the cell-type mean: $+0.011$ nats per UMI).

(b) *K562, no control cell, unseen single gene*: MAML2, the median of 21 units (NLL difference to the additive baseline: -0.015 nats per UMI); Pearson correlation with the real log fold change on its top-20 DE genes: 0.33

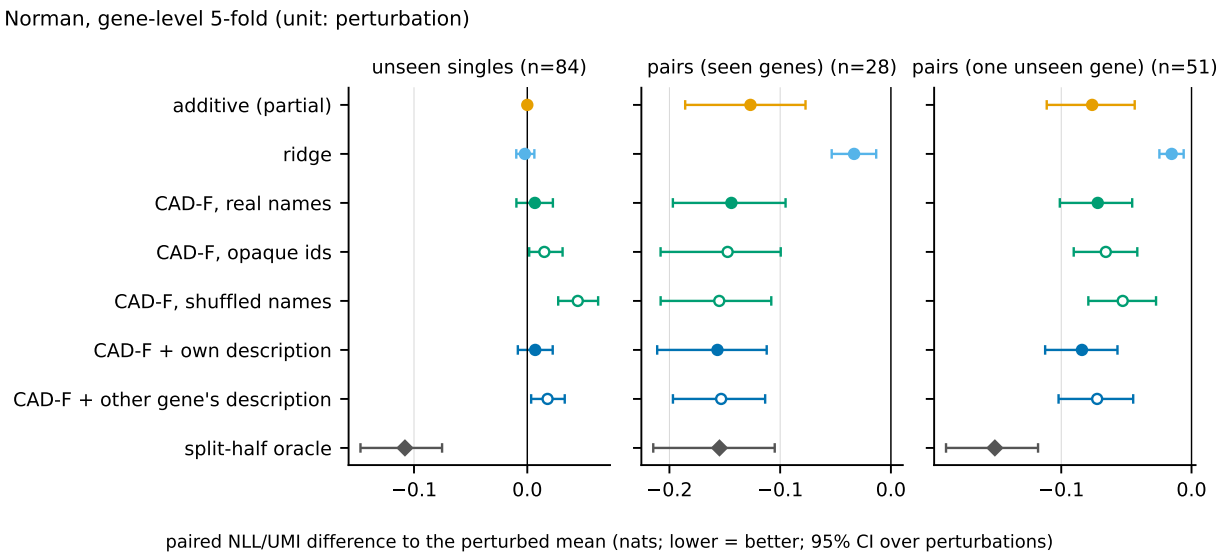


Figure 3: Gene-level 5-fold on Norman, population level: paired held-out NLL difference to the perturbed mean (bars: 95% bootstrap over perturbations; hollow: identity or knowledge controls; diamond: split-half oracle).

Task	Model	ΔNB [95% CI]	Δmultinomial
PBMC (vs. cell-type mean)	global mean	$+0.302$ [$+0.252$, $+0.352$]	$+1.125$
	scVI	$+0.004$ [$+0.001$, $+0.007$]	$+0.005$
	scANVI	$+0.002$ [$+0.000$, $+0.005$]	$+0.004$
	CAD (Gemma)	$+0.004$ [$+0.002$, $+0.006$]	$+0.017$
	CAD (Qwen)	$+0.005$ [$+0.002$, $+0.009$]	$+0.030$
Norman (vs. perturbed mean)	additive	-0.014 [-0.023, -0.004]	-0.094
	ridge	-0.008 [-0.010, -0.006]	-0.024
	scGen-style	-0.014 [-0.021, -0.008]	-0.085
	CAD (Gemma, no control)	-0.021 [-0.027, -0.015]	-0.104
	CAD (Gemma, control)	-0.017 [-0.023, -0.012]	-0.091
	CAD (Qwen, no control)	-0.019 [-0.025, -0.013]	-0.092
	CAD (Qwen, control)	-0.015 [-0.020, -0.010]	-0.078

Table 7: Negative-binomial and multinomial NLL per UMI relative to the reference (test split, seed 0; earlier runs, with control cells not matched across methods; negative favors the model; paired bootstrap over cell types or perturbations).

for CAD and 0.19 for the reference.

(c) *K562, no control cell, pair with one unseen gene*: FOXF1+FOXL2, the median of 29 units (NLL difference to the additive baseline: $+0.011$ nats per UMI); Pearson correlation with the real log fold change on its top-20 DE genes: 0.92 for CAD and 0.96 for the reference.

(d) *K562, no control cell, pair of two seen genes*: DUSP9+SNAI1, the median of 26 units (NLL difference to the additive baseline: -0.012 nats per UMI); Pearson correlation with the real log fold change on its top-20 DE genes: 0.91 for CAD and 0.95 for the reference.

(e) *K562, control cell as text and residual, pair of two seen genes*: ETS2+CNN1, the median of 26 units (NLL difference to the additive baseline: $+0.006$ nats per UMI); Pearson correlation with the real log fold change on its top-20 DE genes: 0.91 for CAD and 0.98 for the reference.

(f) *K562, control cell as residual only, pair of two seen genes*: FOXA1+FOXL2, the median of 26 units (NLL difference to the additive baseline: -0.004 nats per UMI); Pearson correlation with the real log fold change on its top-20 DE genes: 0.87 for CAD and 0.83 for the reference.

(g) *PBMC, zero-shot cell type (fold 0)*: naive thymus-derived CD8-positive, alpha-beta T cell, the median of 4 units (NLL difference to the nearest Cell Ontology type: -0.728 nats per UMI).

Model	NLL ↓	cell NLL ↓	MMD ↓	AUC ↓	r_μ ↑	Type acc. ↑
frequency	**5.629**	**5.629**	0.1508	0.963	0.947	**0.887**
smoothed	5.801 (+0.172 [+0.144, +0.196])	5.801	0.2245	0.994	0.923	0.761
log1p	5.773 (+0.144 [+0.097, +0.199])	5.773	**0.0934**	**0.958**	**0.955**	0.854
sqrt	5.747 (+0.118 [+0.087, +0.152])	5.747	0.0939	0.962	0.955	0.835

Table 8: Human PBMC (Hao et al.), validation split: 19 held-out cell types; CAD with the Qwen3.5-0.8B backbone and four target transformations. NLL: multinomial negative log-likelihood per UMI of the pooled real cells under the model (Delta: paired difference to frequency [95% bootstrap over cell types]). MMD, AUC (real vs generated classifier; 0.5 is ideal), r_μ (gene-mean correlation), type acc.: a classifier trained on real cells recovers the requested type. The generation metrics use cells from the shared Dirichlet-multinomial sampler. Best in bold; our rows shaded.

E Target transformations

Table 8 (validation, 19 cell types, Qwen3.5-0.8B backbone) shows that the transformation trades likelihood for sample quality on PBMC. Frequency gives the best NLL, and smoothing is worse on every metric ($+0.172$ nats, 95% CI $+0.144$ to $+0.196$; MMD $+0.074$). Log1p and square-root targets are $+0.144$ and $+0.118$ nats worse in NLL, yet their MMD is lower (-0.057, CI -0.107 to -0.016, and -0.057, CI -0.109 to -0.014) with unchanged real-versus-generated AUC. A flatter target moves mass to rarely expressed genes, which lowers the likelihood of raw counts while placing generated cells closer to real ones in PCA space.

For perturbation response the trade-off does not appear (Gemma backbone, test, 76 perturbations; Appendix H). Relative to the frequency target, log1p is 0.288 nats worse in NLL (CI $+0.249$ to $+0.333$) and square root 0.250 worse (CI $+0.219$ to $+0.282$), and both also have a higher MMD (0.094 and 0.087 against 0.039) and a higher real-versus-generated AUC (0.875 and 0.871 against 0.811). We therefore keep frequency as the default, and we report the sample-quality advantage of flattened targets as specific to the conditional PBMC task in our experiments.

F UMAP views of generated cells

G Decision rules and the test-split record

Beyond the main configurations, every variant was first run on validation (Qwen3.5-0.8B backbone, seed 0, matched control cells, comparators re-run with the same control cells) and promoted to the test split only if, on validation, its paired 95% interval against its comparator excluded zero in its favor on at least one metric fixed in advance, and no such metric was worse with an interval excluding zero; at most three new configurations were promoted. Promoted configurations were then run once per seed (0, 1, 2) on test with the Gemma backbone. The rules and decisions were written to the repository before the test runs.

Variant	Comparator	Primary metric(s)	Decision
Control residual, no control text	control residual with mass text	per-cell NLL	beat comparator; test, 3 seeds
Control text, no residual	control residual with mass text	per-cell NLL	worse; test, 1 seed (ablation)
Contrast control text	control residual with mass text	per-cell NLL, MMD, r_Δ	promoted
Gene summaries in the prompt	other gene's summary; opaque ids	NLL, r_Δ on unseen singles	not promoted
UMI-weighted loss	same model, equal weights	NLL, r_Δ, r_{DE}	not promoted
Negative-binomial head	softmax head	NLL, r_Δ, r_{DE}	promoted (Norman only)
Low-rank head	softmax head	per-cell NLL, MMD, AUC	promoted (no control cell)
Zero-shot cell types	nearest Cell Ontology type	NLL	passed (gate G3)
C2S-Gemma mixing weight	—	validation NLL	0.9 (grid maximum)

Table 9: Validation decisions taken before the test runs.

Four analyses were added after the test results had been seen, and are labeled as such where they appear: the R^2 of the rank reconstruction (Section 5.1), the control-cell residual without a language model (Section 5.3; a first version

Model	NLL ↓	cell NLL ↓	MMD ↓	AUC ↓	r_μ ↑
frequency	**5.186**	**5.186**	**0.0393**	**0.811**	**0.957**
log1p	5.474 (+0.288 [+0.249, +0.333])	5.474	0.0941	0.875	0.938
sqrt	5.435 (+0.250 [+0.219, +0.282])	5.435	0.0869	0.871	0.940

Table 10: Norman K562, test split: 76 held-out perturbations; CAD (Gemma), no control cell, with three target transformations. NLL: multinomial negative log-likelihood per UMI of the pooled real cells under the model (Delta: paired difference to frequency [95% bootstrap over perturbations]). cell NLL: per-cell mixture over M random control cells. MMD, AUC (real vs generated classifier; 0.5 is ideal), r_μ (gene-mean correlation). The generation metrics use cells from the shared Dirichlet-multinomial sampler. Best in bold; our rows shaded.

Model	NLL ↓	cell NLL ↓	MMD ↓	AUC ↓	r_μ ↑
mass text	5.199	5.173	**0.0371**	0.816	**0.955**
rank text	**5.192 (-0.007 [-0.012, +0.000])**	**5.159**	0.0375	**0.809**	0.954

Table 11: Norman K562, test split: 76 held-out perturbations; CAD (Gemma), control residual, with the control cell given as mass levels or as a 200-gene rank sentence. NLL: multinomial negative log-likelihood per UMI of the pooled real cells under the model (Delta: paired difference to mass text [95% bootstrap over perturbations]). cell NLL: per-cell mixture over M random control cells. MMD, AUC (real vs generated classifier; 0.5 is ideal), r_μ (gene-mean correlation). The generation metrics use cells from the shared Dirichlet-multinomial sampler. Best in bold; our rows shaded.

that divided by the pooled control frequency instead of the mean prior was not a fair analogue and was discarded), the name-overlap and Cell Ontology parent baselines for zero-shot types (Section 5.4), and the decoding-speed benchmark (Section 5.6). Each can only weaken our claims.

Deviations, all disclosed: a 128-gene mass-level control text was not evaluated because it repeatedly ran out of GPU memory; three Qwen control-residual test runs ran out of memory during the test lanes and were re-run alone with the same configuration; and the zero-shot gate was applied on the 8 types of folds 0 and 1 because three validation runs of folds 2–4 had run out of memory, which were then re-run (on all 19 types the gate gives the same decision, -0.476 nats per UMI, CI -0.971 to -0.123).

H Additional tables

The tables in this section and in Appendix E come from the earlier runs, in which the control cells were drawn separately for each method; the main-text tables use matched control cells. Table 10 (test) compares the target transformations for perturbation response with the Gemma backbone, and Table 11 (test) compares the two control-cell formats. Tables 12 and 13 repeat the comparison with the other generators on the validation split, on which all decisions were made; Table 14 shows the LoRA-rank sensitivity on Norman (validation).

I Proofs

Observation 1. (i) $x = Np$ recovers the counts from (N, p), and (N, p) is a function of x. Under $\mathrm{Mult}(N, q)$, $\log P(x) = \log \frac{N!}{\prod_g x_g!} + \sum_g x_g \log q_g = \text{const} + N \sum_g p_g \log q_g$, which by Gibbs' inequality is maximized over $q \in \Delta$ at $q = p$. (ii) For any x with at least two distinct nonzero counts, x and $2x$ have the same ordering and hence the same sentence, and so do x and any count vector obtained by a strictly increasing change of the counts. (iii) x and $2x$ have the same binarization. For $f(t) = \log(1 + t)$ or $f(t) = \sqrt{t}$, the renormalized vector is $p'_g = f(x_g)/\sum_h f(x_h)$, and $p' = p$ requires $f(x_g)/x_g$ to be equal for all genes with $x_g > 0$; because $f(t)/t$ is strictly decreasing for $t > 0$, this holds only when all nonzero counts are equal.

Model	NLL ↓	cell NLL ↓	MMD ↓	AUC ↓	r_μ ↑	Type acc. ↑
Global mean	6.700 (+1.073 [+0.892, +1.283])	6.700	0.8462	1.000	0.565	0.048
Cell-type mean	**5.627**	**5.627**	0.1688	0.983	0.942	**0.928**
scVI	5.634 (+0.007 [-0.005, +0.018])	5.634	0.1576	0.968	0.946	0.906
scANVI	5.630 (+0.003 [-0.007, +0.012])	5.630	0.1494	0.967	0.947	0.878
CAD (Gemma)	5.628 (+0.001 [-0.007, +0.008])	5.628	**0.1440**	0.972	**0.948**	0.887
CAD (Qwen), seed 1	5.632 (+0.005 [-0.003, +0.011])	5.632	0.1464	**0.963**	0.947	0.900

Table 12: Human PBMC (Hao et al.), validation split: 19 held-out cell types; CAD rows: first seed unless stated; baselines: one run each. NLL: multinomial negative log-likelihood per UMI of the pooled real cells under the model (Delta: paired difference to Cell-type mean [95% bootstrap over cell types]). MMD, AUC (real vs generated classifier; 0.5 is ideal), r_μ (gene-mean correlation), type acc.: a classifier trained on real cells recovers the requested type. The generation metrics use cells from the shared Dirichlet-multinomial sampler. Best in bold; our rows shaded.

Model	NLL ↓	cell NLL ↓	MMD ↓	AUC ↓	r_μ ↑
Perturbed mean	5.314	5.314	0.0779	0.915	0.920
Additive	5.248 (-0.066 [-0.096, -0.040])	5.248	0.0459	0.838	0.952
Ridge	5.306 (-0.008 [-0.021, +0.004])	5.306	0.0670	0.874	0.936
scGen-style	5.271 (-0.043 [-0.072, -0.017])	**5.162**	**0.0300**	**0.774**	0.955
CAD (Gemma), no control cell	**5.235 (-0.079 [-0.105, -0.059])**	5.235	0.0360	0.799	**0.959**
CAD (Gemma), control residual, mass text	5.246 (-0.068 [-0.094, -0.046])	5.221	0.0393	0.824	0.952
CAD (Gemma), control residual, rank text	5.240 (-0.075 [-0.099, -0.054])	5.209	0.0370	0.821	0.954
CAD (Qwen), no control cell, seed 1	5.239 (-0.076 [-0.102, -0.055])	5.239	0.0400	0.823	0.955

Table 13: Norman K562, validation split: 48 held-out perturbations; CAD rows: first seed unless stated; baselines: one run each. NLL: multinomial negative log-likelihood per UMI of the pooled real cells under the model (Delta: paired difference to Perturbed mean [95% bootstrap over perturbations]). cell NLL: per-cell mixture over M random control cells. MMD, AUC (real vs generated classifier; 0.5 is ideal), r_μ (gene-mean correlation). The generation metrics use cells from the shared Dirichlet-multinomial sampler. Best in bold; our rows shaded.

Throughout, Δ is the simplex over the G panel genes and modules $S_1, \dots, S_M$ partition the genes; P1, P3 and P4 concern the hierarchical head of Appendix J, and P2 concerns the NLL used throughout.

P1 (normalization and exact factorization). For any $q^{\mathrm{root}} \in \Delta^M$ and $q^m \in \Delta^{|S_m|}$, $q(g) = q^{\mathrm{root}}(m(g))\, q^{m(g)}(g)$ lies in Δ, and for counts X with module totals n_m,

$$-\sum_g X_g \log q(g) = -\sum_m n_m \log q^{\mathrm{root}}(m) - \sum_m \sum_{g \in S_m} X_g \log q^m(g).$$

Every q in the interior of Δ factors uniquely this way, so the hierarchical head is a *re-parametrization* of the flat head: it changes what the network must output, not the family of distributions it can represent.

P2 (the metric is proper). If the counts X of a perturbation's cells satisfy $\mathbb{E}[X_g / \sum_h X_h] = \pi_g$ given the total, the expected NLL per UMI of q is $H(\pi) + \mathrm{KL}(\pi \,\|\, q)$ and is uniquely minimized at $q = \pi$. Differences in expected NLL between two decoders therefore equal differences in $\mathrm{KL}(\pi \| q)$.

P3 (tilt identity and its population minimizer). For $f, \tilde{p}$ in the interior of Δ, $q_g \propto f_g t_g$ equals $\tilde{p}_g$ exactly. If a context is paired with bags b having tilts t_b, the minimizer of the expected soft cross-entropy is $\mathbb{E}[t_b]$, so the implied decoder is $q_g \propto \mathbb{E}_b[\tilde{p}_{b,g}/Z_b]$: a $1/Z_b$-weighted average of bag frequencies, which equals the plain average whenever Z_b is constant within a context. The target is the shrunk $\tilde{p}$, so $\|\tilde{p} - p\|_1 \le \frac{\lambda}{n+\lambda}\|f - p\|_1 \le 2\frac{\lambda}{n+\lambda}$ ($\approx 1\%$ for $n \approx 2\times 10^5$ UMIs per bag).

P4 (sampled-leaf loss). Let $T_m = \sum_{g \in S_m} t_g$. With modules $A = (m_1, \dots, m_R)$ drawn i.i.d. from T, $\mathrm{CE}(T, r^{\mathrm{root}}) + \mathbb{E}\big[\frac{1}{R}\sum_r \mathrm{CE}(t|_{m_r}, r^{m_r})\big] = \mathrm{CE}(t, r)$, the full tree cross-entropy. *Our implementation samples without replacement and the trainer averages questions with equal weight, so its record loss is a weighted surrogate* $(1/(R{+}1)$

Model	NLL ↓	cell NLL ↓	MMD ↓	AUC ↓	r_μ ↑
rank 16	**5.235**	**5.235**	**0.0360**	**0.799**	**0.959**
rank 8	5.237 (+0.002 [-0.002, +0.007])	5.237	0.0365	0.812	0.959
rank 64	5.237 (+0.002 [-0.002, +0.007])	5.237	0.0386	0.809	0.957

Table 14: Norman K562, validation split: 48 held-out perturbations; CAD (Gemma), no control cell, with LoRA rank 16, 8 and 64. NLL: multinomial negative log-likelihood per UMI of the pooled real cells under the model (Delta: paired difference to rank 16 [95% bootstrap over perturbations]). cell NLL: per-cell mixture over M random control cells. MMD, AUC (real vs generated classifier; 0.5 is ideal), r_μ (gene-mean correlation). The generation metrics use cells from the shared Dirichlet-multinomial sampler. Best in bold; our rows shaded.

per question), not this unbiased estimator; the two share their minimizer only when every conditional is exactly representable (the well-specified regime).

P1. $\sum_g q(g) = \sum_m q^{\mathrm{root}}(m) \sum_{g \in S_m} q^m(g) = \sum_m q^{\mathrm{root}}(m) = 1$ and every factor is nonnegative. The loss identity follows from $\log q(g) = \log q^{\mathrm{root}}(m(g)) + \log q^{m(g)}(g)$ and $\sum_{g \in S_m} X_g = n_m$. Conversely, for q in the interior set $q^{\mathrm{root}}(m) = \sum_{g \in S_m} q(g)$ and $q^m(g) = q(g)/q^{\mathrm{root}}(m)$; these are the only factors that reproduce q because $q^{\mathrm{root}}(m) = \sum_{g \in S_m} q(g)$ is forced by summing.

P2. Conditional on the total N, $\mathbb{E}[-\sum_g X_g \log q_g / N] = -\sum_g \pi_g \log q_g = H(\pi) + \mathrm{KL}(\pi \| q)$, and $\mathrm{KL} \geq 0$ with equality if and only if $q = \pi$ (Gibbs' inequality). Two decoders therefore differ in expected NLL by $\mathrm{KL}(\pi \| q_A) - \mathrm{KL}(\pi \| q_B)$. Per-UMI scoring weights each perturbation's cells by their UMIs; the paired bootstrap treats the perturbation as the unit.

P3. $f_g t_g = \tilde{p}_g / Z$, so $\sum_h f_h t_h = 1/Z$ and $f_g t_g / \sum_h f_h t_h = \tilde{p}_g$. For targets t_b drawn with context c, $\mathbb{E}_b[\mathrm{CE}(t_b, r)] = H(\mathbb{E} t_b) + \mathrm{KL}(\mathbb{E} t_b \| r) + \text{const}$ is minimized by $r = \mathbb{E}[t_b]$, so $q_g \propto f_g \mathbb{E}[t_{b,g}] = \mathbb{E}[\tilde{p}_{b,g}/Z_b]$, a $1/Z_b$-weighted mean of $\tilde{p}_b$. With $\tilde{p} = (1-w)p + wf$ and $w = \lambda/(n+\lambda)$, $\|\tilde{p} - p\|_1 = w\|f - p\|_1 \leq 2w$.

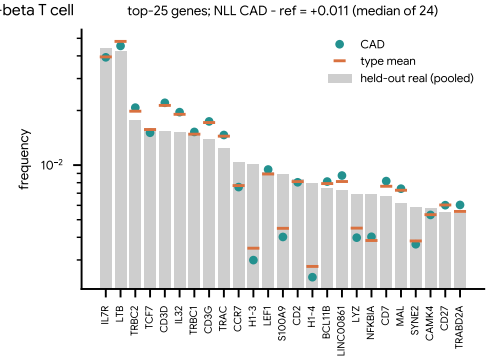
P4. The chain rule gives $\mathrm{CE}(t, r) = \mathrm{CE}(T, r^{\mathrm{root}}) + \sum_m T_m \mathrm{CE}(t|_m, r^m)$ with $t|_m = t_{S_m}/T_m$. For $m_1, \ldots, m_R$ i.i.d. from T, $\mathbb{E}[\mathrm{CE}(t|_{m_r}, r^{m_r})] = \sum_m T_m \mathrm{CE}(t|_m, r^m)$ for each r, which gives the claim. Sampling without replacement changes the inclusion probabilities away from T_m, and averaging over the $R{+}1$ answered questions gives each question weight $1/(R{+}1)$ instead of $(1, \frac{1}{R}, \ldots, \frac{1}{R})$; if every conditional is representable, each term is minimized at the true conditional and the minimizer is unchanged.

J Hierarchical head ablation

The hierarchical variant (CAD-N/CAD-R) answers one 64-option module question and four 32-option gene questions through the Unsloth decision head, trained on a *tilt* target (the ratio of the perturbed to the control frequency, renormalized, with the control prior f multiplied back at generation). On the corrected Norman split, trained on absolute frequencies it is $+0.161$ [$+0.150$, $+0.174$] nats worse than the perturbed mean (control-state NLL 5.490 against 5.295 for the control frequencies); the tilt repairs the fit (control 5.300; tilt minus no-tilt -0.138 [-0.157, -0.119]) but the tilted head remains $+0.023$ [$+0.009$, $+0.038$] worse than the perturbed mean and 0.100 nats worse than the flat head. The biological partition beats a random one by 0.019 ([-0.035, -0.002]). We therefore use the flat head in the main text.

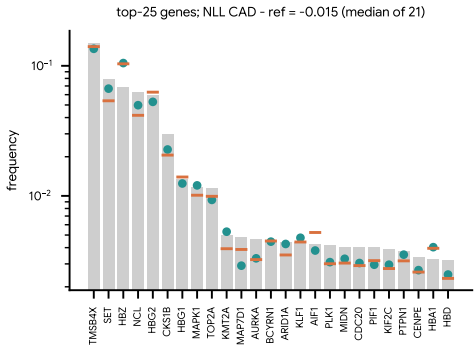
PBMC, conditional generation
unit: naive thymus-derived CD4-positive, alpha-beta T cell

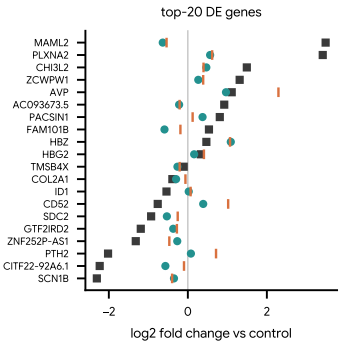
```
Cell context: human peripheral blood
mononuclear cells (PBMC).
Cell type: naive thymus-derived CD4-positive,
alpha-beta T cell.
Gene expression frequency distribution of the
cell:
```



K562, no control cell, unseen single gene
unit: MAML2

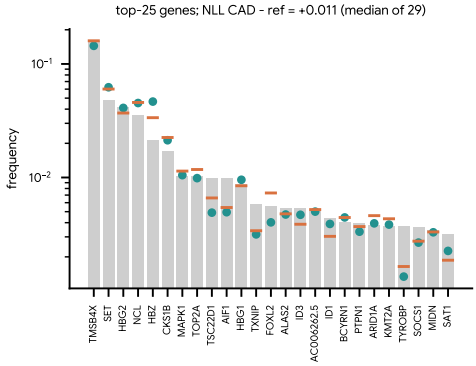
```
Cell context: K562 cells, CRISPRa
perturbation.
Perturbation: MAML2.
Gene expression frequency distribution of the
perturbed cell:
```

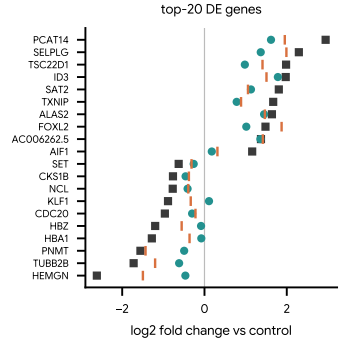




K562, no control cell, pair with one unseen gene
unit: FOXF1+FOXL2

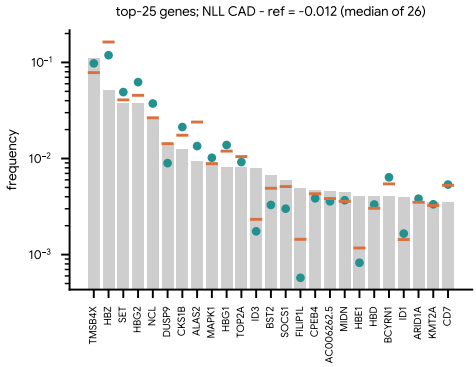
```
Cell context: K562 cells, CRISPRa
perturbation.
Perturbation: FOXF1+FOXL2.
Gene expression frequency distribution of the
perturbed cell:
```

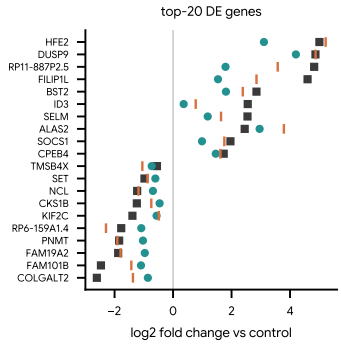




K562, no control cell, pair of two seen genes
unit: DUSP9+SNAI1

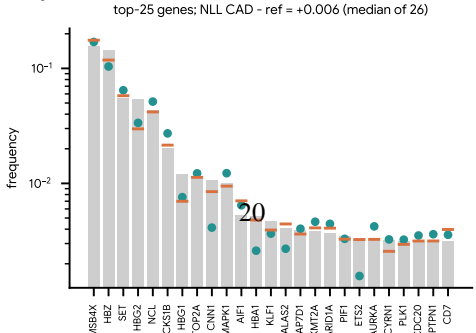
```
Cell context: K562 cells, CRISPRa
perturbation.
Perturbation: DUSP9+SNAI1.
Gene expression frequency distribution of the
perturbed cell:
```

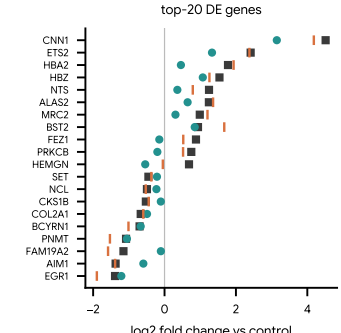




K562, control cell as text and residual, pair of two seen genes
unit: ETS2+CNN1

```
Cell context: K562 cells, CRISPRa
perturbation.
Perturbation: ETS2+CNN1.
Control cell (top genes with log-mass level):
TMSB4X:10 HBZ:9 SET:9 HBG2:8 CKS1B:8 NCL:8
KLF1:8 HBD:7 CD7:7 MAPK1:6 SMIM1:6 PLK1:6
MIDN:6 IL6ST:6 COL2A1:6 KMT2A:6 TSC22D1:5
ALAS2:5 SPI1:5 TAS2R14:5 ASCL2:5 RP6-159A1.4:5
HGSNAT:5 HIST2H2AC:5 ADSSL1:5 PNMT:5 MAP7D1:5
PIF1:5 ARID1A:5 AURKA:5 SAMD1:5 PTPN1:5
GS1-124K5.11:5 RP11-481J2.4:5 PLK4:5 GDPD1:4
TOP2A:4 BCAS3:4 PYY:4 NBR1:4 ACAP1:4
CTD-2095E4.5:4 COTL1:4 RP13-516M14.1:4
PMAIP1:4 IGFBP4:4 STIL:4 KIF18B:4
Gene expression frequency distribution of the
perturbed cell:
```





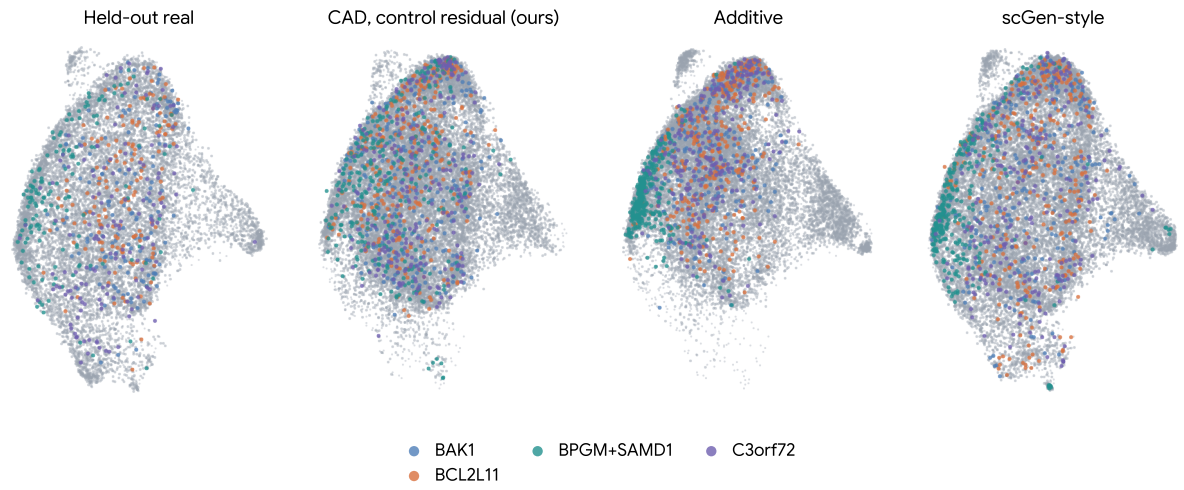


Figure 5: UMAP of held-out real perturbed K562 cells (test split) and of cells generated from the matched control cells by CellAsDistribution with the control-cell residual (Gemma, seed 0), by the additive baseline and by the scGen-style latent shift, all drawn with the shared multinomial sampler. PCA and UMAP are fit on real cells only, and generated cells are projected. Colors: the four most frequent held-out perturbations; grey: the others.

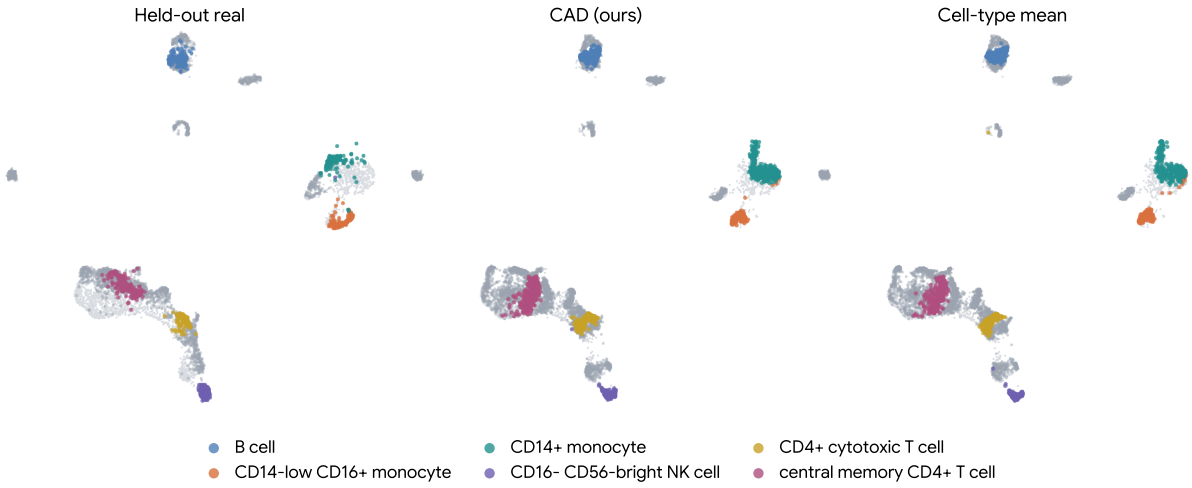


Figure 6: UMAP of held-out real PBMC cells (unseen donors, test split) and of cells generated by CellAsDistribution (Gemma, seed 0) and by the cell-type mean with the same sampler. PCA and UMAP are fit on real cells only, and generated cells are projected. Colors: the six most frequent cell types; grey: the others.